Session 1: AI for Coding — Tools, Concepts, and Setup

Elira Kuka & Tanner Regan

The George Washington University

What We Will Cover (Sessions 1 & 2)

- Part 1: Tools, concepts, and setup
  - Overview of AI coding tools and how to choose among them
  - How to use AI Agents: Terminal, Desktop App, VS Code
  - Conceptual overview of context engineering with Claude
  - VS Code and Claude Code integration
  - Git and Gihub: why they matter for research and Claude

What We Will Cover (Sessions 1 & 2)

- Part 1: Tools, concepts, and setup
  - Overview of AI coding tools and how to choose among them
  - How to use AI Agents: Terminal, Desktop App, VS Code
  - Conceptual overview of context engineering with Claude
  - VS Code and Claude Code integration
  - Git and Gihub: why they matter for research and Claude
- Part 2: Practical workflows
  - Writing, debugging, and automating code with AI
  - “Vibe coding” from plain English descriptions
  - Project organization and replication packages
  - Demo: data cleaning script from scratch (python and stata)

Coding Perhaps Highest-Value Use of AI

- AI most reliable when output is verifiable: code runs or it does not
- Economists spend enormous time on repetitive coding tasks
  - AI can handle most of them
- Massive productivity gains across skill levels:
  - Beginners: “vibe code” to write code in an unfamiliar language (no syntax!)
  - Advanced users: automate drudge work, prototype faster, reduce bugs

Coding Perhaps Highest-Value Use of AI

- AI most reliable when output is verifiable: code runs or it does not
- Economists spend enormous time on repetitive coding tasks
  - AI can handle most of them
- Massive productivity gains across skill levels:
  - Beginners: “vibe code” to write code in an unfamiliar language (no syntax!)
  - Advanced users: automate drudge work, prototype faster, reduce bugs

$\Rightarrow$ First two sessions will cover how to use AI to complete coding tasks

Overview of AI Coding Tools

The AI Coding Landscape: Many Options

- General-purpose AI assistants (good at coding, among many other things):
  - **Claude** (Anthropic) — our focus today
  - **ChatGPT / GPT-4o** (OpenAI)
  - **Gemini** (Google)
- Agentic coding tools (read files, write and run code autonomously):
  - **Claude Code** (Anthropic) — our focus today
  - **Codex** (OpenAI) — closest competitor to Claude Code
  - **GitHub Copilot** (Microsoft) — inline suggestions in editor
  - **Cursor / Windsurf** — AI-first code editors
- How to choose: task type, privacy needs, and workflow integration

Chatbot vs. Agentic Tool

- **Chatbot** (Claude.ai, ChatGPT): you ask → it answers → *you* act
- **Agentic tool** (Claude Code): you give a goal → it reads files, writes code, runs it, fixes errors, iterates → reports back
- For economists: instead of "here is some code," it runs the code and tells you what it found

Which One Should You Use? Claude Code vs. Codex

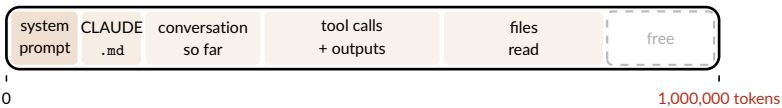
- Two serious options for agentic coding today: **Claude Code** and **Codex**
- Plans mirror each other at each price point:
  - $20/mo: Claude Pro vs. ChatGPT Plus
  - $100/mo: Claude Max (5x) vs. ChatGPT Pro (5x)
  - $200/mo: Claude Max (20x) vs. ChatGPT Pro (20x)
- Both use a 5-hour rolling window + weekly cap; limits reset automatically
- At the same price, Codex currently gives more usage per dollar

Which One Should You Use? Claude Code vs. Codex

- Two serious options for agentic coding today: **Claude Code** and **Codex**
- **Codex** has some advantages:
  - More usage per subscription dollar — important for heavy users
  - Sometimes stronger on pure coding benchmarks
- **Claude Code** has some advantages:
  - Often better at tasks beyond code: understanding context, writing, reasoning
- **Bottom line:** this comparison is fast-moving and will keep changing
  - I am using Claude Code and find it excellent for my workflow
  - I would switch only if Codex proved clearly and consistently superior
  - My advice: pick one, learn it well, reassess in 6 months

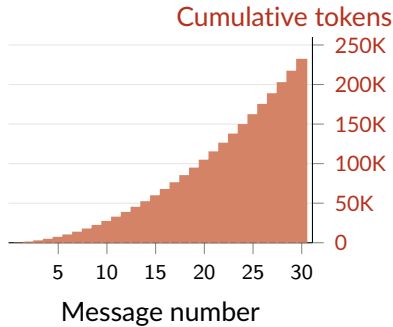
What You Pay For: Tokens and Context

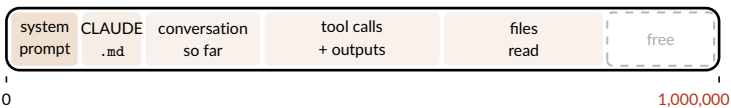
- **Token**: the unit AI reads and writes — roughly $\frac{3}{4}$ of a word
  - **You pay for**: tokens *in* (prompt + files) and tokens *out* (reply) — input is cheaper
- **Context window**: everything the model holds at once — fills as session grows



What You Pay For: Tokens and Context

- **Token**: the unit AI reads and writes — roughly $\frac{3}{4}$ of a word
  - **You pay for**: tokens *in* (prompt + files) and tokens *out* (reply) — input is cheaper
- **Context window**: everything the model holds at once — fills as session grows





Getting the Most Out of Your Subscription

- Subscriptions have rolling limits — fewer tokens per task means more use
- How to use fewer tokens:
  - Start a new session for each new task: context accumulates and raises costs
  - Store standing instructions once so you never repeat yourself
  - Be specific: vague prompts waste tokens and invite unfocused replies
  - Ask multiple things at once so less back and forth
  - Compact your chats/context window
- Silly trick: start Claude when wake up so 5-hour window ends earlier in day

Claude Code / Codex Interfaces

Three Ways to Use an Agentic Coding Tool

- **Terminal** (command line):
  - Most powerful; works directly in your project directory
  - Best for: running scripts, data analysis, replication tasks
- **Desktop App**:
  - Friendlier interface; easier to get started
  - Projects feature: persistent context across sessions
  - Best for: drafting, quick questions, early exploration
- **VS Code integration**:
  - Agent lives inside your editor — see code and AI side by side
  - Best for: sustained development work, Git integration, Jupyter/Python

Terminal vs. VS Code: Quick Comparison

Terminal (CLI)

- Lightweight; fast startup
- Works anywhere, incl. remote servers
- Keyboard-driven; fast for quick tasks
- No inline diffs; steeper setup

VS Code Extension

- Inline diffs; accept changes block-by-block
- Code & AI panel side by side
- Best for large codebases, sustained work
- Heavier on CPU and memory

Live Demo: Terminal

- ▷ *Demo slide — minimal text, walk through live*
- Navigate to a project directory
- Ask Claude Code to describe what it sees
- Ask it to write a simple data task

Live Demo: Desktop App

- ▷ *Demo slide — minimal text, walk through live*
- Create a Project and load context
- Same task as terminal — compare experience

Live Demo: VS Code

- ▷ *Demo slide — walk through VS Code interface live*
- Same task as terminal — compare experience

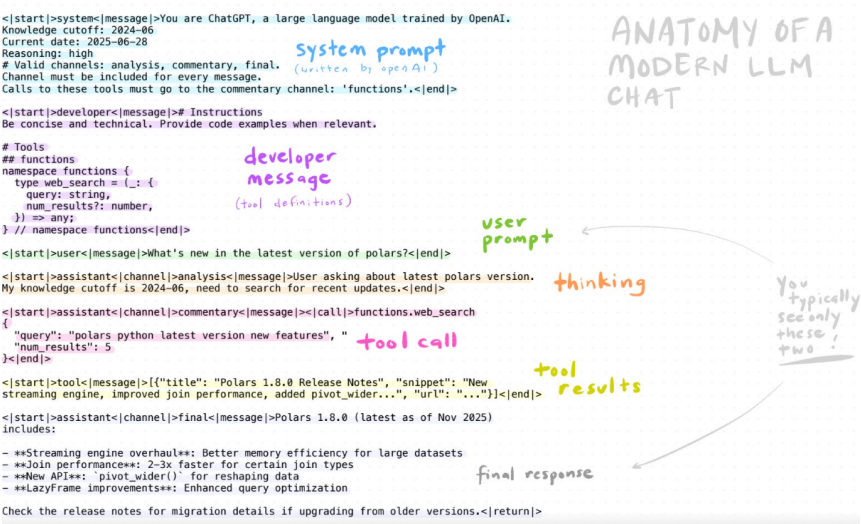
Contex Engineering with Claude Overview

Contex Engineering

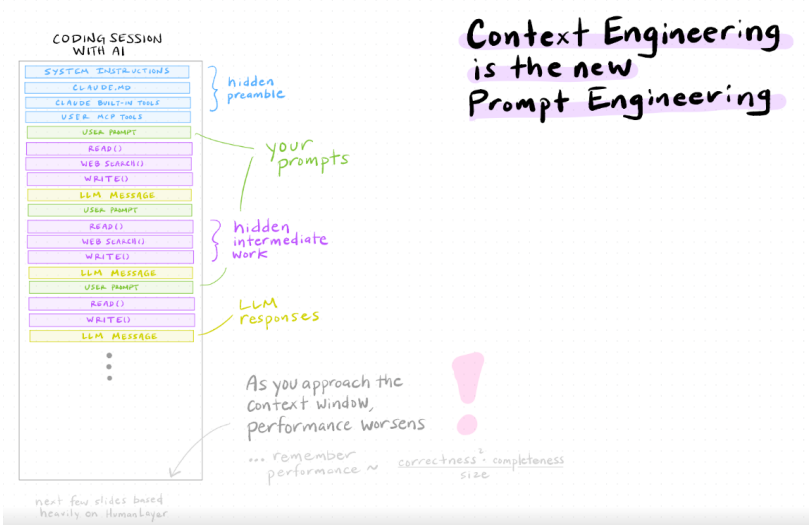
- Context Window
  - Each prompt to Claude Code bundles the entire history of your session
    - your prompts, Claude's responses, files it read, code outputs, system promtsz
  - These are all appended until the context window hits it's fixed size limit
    - For Claude Code this is around 200k tokens
  - Claude then "**auto-compacts**" writes it's own summary and then restarts fresh
    - less preferable to frequent intentional compaction
- Contex Engineering
  - You can tell claude when to compact using `/compact your instructions`
  - Develop habits that the context window focussed on relevant information
    - Claude.md vs skills —and— 'ask before edits' vs 'plan' Modes

Read more at Paul GP's Guide

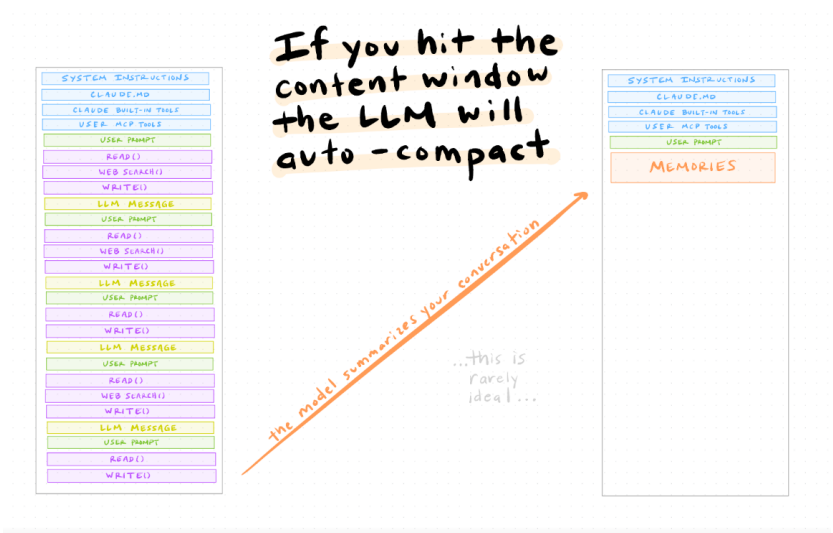
Anatomy of a context window (example from Paul GP's Guide)



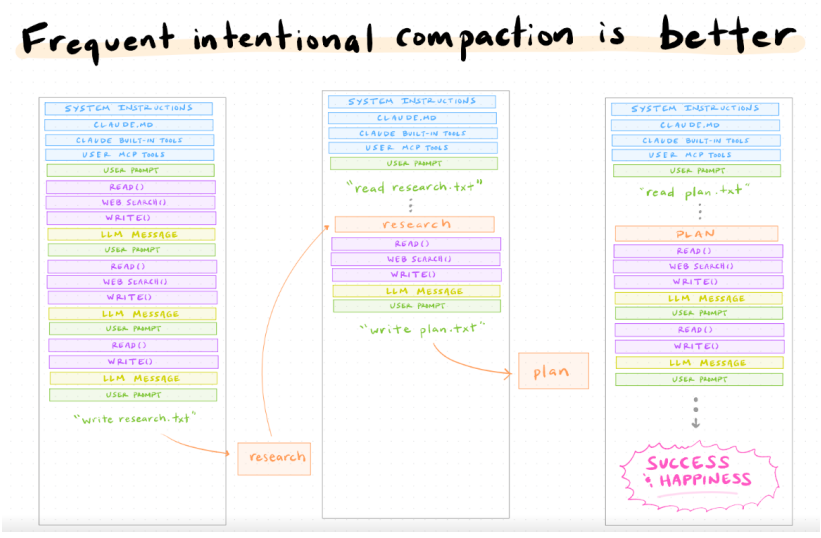
Context engineering (example from Paul GP's Guide)



Context auto-compact (example from Paul GP's Guide)



Context intentional compacting (example from Paul GP's Guide)



CLAUDE.md and Skills

- **CLAUDE.md** (memory) — Claude reads at the start of every session
  - Store context that is always true so you don't repeat yourself in every prompt
  - `.claude/CLAUDE.md` is global, `your-project/CLAUDE.md` is *project* specific
  - Ask Claude to update after substantial changes to project
- **Skills** — Claude reads every time it is called
  - Store context for a task that you repeat often, but not always relevant
  - `.claude/skills/your-skill/SKILL.md` is called with `/your-skill`
  - Write and update your own skills or download from others

Read more at Claes Backman's Guide (full descriptions, examples of CLAUDE.md and skills, download pre-made skills)

`your-project/CLAUDE.md` (example from Claes Backman's Guide)

```
# Project: [Paper title]

## Data
- Unit of analysis: firm-year panel, 2010–2022
- Raw data lives in data/raw/ — never edit these files
- Main dataset after cleaning: data/clean/panel.dta (N ≈ 47,000)

## Code conventions
- R scripts are numbered (01_clean.R, 02_analysis.R, ...)
- All regressions cluster SEs at the firm level
- Use fixest for all panel regressions

## LaTeX
- Main file: paper/main.tex
- Tables generated by 03_tables.R go to paper/tables/
- Use \widehat{} not \hat{} for estimators
```

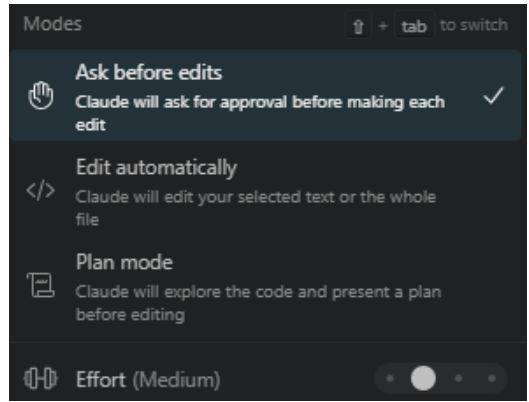
`.claude/skills/robustness/SKILL.md` (example from Claes Backman's Guide)

```
---
name: robustness
description: Run standard robustness checks on the main regression.
---

For the main specification in this project:
1. Re-run with alternative clustering (industry-year instead of firm)
2. Re-run dropping the top and bottom 1% of the outcome variable
3. Re-run on the pre-2020 subsample only
4. Produce a summary table comparing coefficients across all specs
```

Claude Modes

- **Ask before edits**
  - Do steps one at a time
  - Asks your permission before running each and every command
- **Edit automatically**
  - Same as above but runs commands without asking permission (risky)
- **Plan**
  - Claude writes plan.md with many steps
  - You review and execute entire plan



Other tips for managing the context window

- **Use @-mentions to be selective**
  - Rather than Claude reading everything, point to a specific file
  - e.g. `update all the pathnames in @Lecture_2.tex`
- **Check usage with /context**
  - Type /context in the panel to see a breakdown of how your context window is being used and how much space remains.

VS Code with Claude Overview

What is VS Code?

- Integrated Development Environment (IDE)
  - Free, open source (created by Microsoft), and very popular
- Core features:
  - **Explorer**: file browser and project tree
  - **Editor**: syntax highlighting for every language (Stata, R, Python, LaTeX, …)
  - **Integrated terminal**: run scripts without leaving the editor
  - **Extensions**: Git, Claude Code, Jupyter, and more
  - **Command Palette** (`Ctrl+Shift+P`): search and run any action by name
- Claes Backman's Guide Claude Code in VS Code for Academic Economists
  - Other resources: Official intro videos (Microsoft), Tips and Tricks (Microsoft)

File Edit Selection View Go Run Terminal Help

Untitled (Workspace)

Lecture_1.tex 7, M | _preamble.tex

les > Lecture_1.tex > \textcolor{violet}{VS Code with Claude Overview \vspace{0.3in}} > { } Frame: VS Code with Claude Overview

claesbackman Aa ab .* 3 of 5

```
\section{\textcolor{violet}{VS Code with Claude Overview \vspace{0.3in}}}

%%%%%%%%%%%%%%%
\begin{frame}
\frametitle{What is VS Code?}

\begin{itemize}
    \item Integrated Development Environment (IDE)
    \begin{itemize}
        \item Free, open source (created by Microsoft), and very popular
    \end{itemize}
    \item Core features:
        \begin{itemize}
            \item[$\circ$] \textbf{Explorer}: file browser and project tree
            \item[$\circ$] \textbf{Editor}: syntax highlighting for every langua
            \item[$\circ$] \textbf{Integrated terminal}: run scripts without lea
            \item[$\circ$] \textbf{Extensions}: Git, Jupyter, Claude Code, and m
            \item[$\circ$] \textbf{Command Palette} (\hlcode{Ctrl+Shift+P}): sea
        \end{itemize}
    \vspace{.05in}
    \item Resources: \href{https://code.visualstudio.com/docs/introvideos/basics
\end{itemize}
\end{frame}


%%%%%%%%%%%%%%%
\begin{frame}[plain]
\begin{center}
    \includegraphics[width=\paperwidth, height=\paperheight, keepaspectratio=fal
\end{center}
\end{frame}



%%%%%%%%%%%%%%%
\begin{frame}
\frametitle{VS Code with Claude Overview}

\begin{itemize}
    \item Key panels: Explorer (files), Editor, Terminal, Source Control (Git),
```

Lecture_1.pdf M

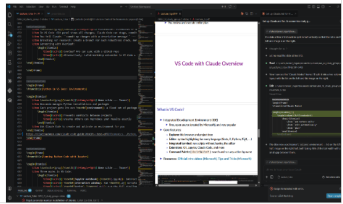
GWU_AI_study_group > slides > Lecture_1.pdf

27 / 40

What is VS Code?

- Integrated Development Environment (IDE)
  - Free, open source (created by Microsoft), and very popular
- Core features:
  - **Explorer**: file browser and project tree
  - **Editor**: syntax highlighting for every language (Stata, R, Python, LaTeX, …)
  - **Integrated terminal**: run scripts without leaving the editor
  - **Extensions**: Git, Jupyter, Claude Code, and more
  - **Command Palette** (`Ctrl+Shift+P`): search and run any action by name
- Resources: Official intro videos (Microsoft), Tips and Tricks (Microsoft)

28 / 40



VS Code with Claude Overview

Set up Claude.md for AI ...

Set up Claude.md for AI economics study g...

c:\Users\tanner_regan\Docu...

the slide at line 510 should be split in half vertically so that the list is on th
left and image is on the right.

- Thought for 1s
- Let me read the slide at line 510.
- **Read** `c:\Users\tanner_regan\Documents\GitHub\GWU_AI_study_group\sl`
  `es\Lecture_1.tex` (lines 501-540)
- Now I can see the "Claude Modes" frame. I'll split it into a two-column
  layout with the list on the left and the image on the right.
- **Edit** `c:\Users\tanner_regan\Documents\GitHub\GWU_AI_study_group\sli`
  `s\Lecture_1.tex`

  Added 4 lines

```
%%%%%%%%%%%%%%%
\begin{frame}
\frametitle{Claude Modes}

\begin{columns}[T]
  \begin{column}{0.48\textwidth}
    \begin{itemize}
      \item `ask before edits'
      \item `edit automatically'
      \item `plan'
    \end{itemize}
  \end{column}
```

- The slide now uses Beamer's `columns` environment — list on the left
  half, image on the right half, both taking 48% of the text width with a
  small gap between them.

c:\Users\tanner_regan\Docu...

ctrl esc to focus or unfocus Claude

Lecture_1.tex | Ask before edits

PROBLEMS 11 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

Filter (e.g. text, **/*.ts, !**/node...

Lecture_1.tex GWU_AI_study_group • slides 10

⚠ Package hyperref: Option `pdfauthor' has already been used. LaTeX [Ln 8, Col 1]

Claude Code **extension** in VS Code

- Each new conversation in the Claude panel is a fresh context
  - Click the + button at the top of the Claude panel to start one,
  - or type `/clear` to reset the current conversation
- Go over a fresh window $\longrightarrow$
  - Session Title, Session History, Mode menu, Current File Selection, Attachments, Command Menu

What Claude Code Can Read — and What to Convert First

Git: Version Control for Economists

Why Git Matters for Research

- Version control = a complete history of every change to your code
- No more: `analysis_v3_FINAL_revised2.do`
- Enables:
  - Rolling back to any prior version of your code
  - Working safely with coauthors and RAs without overwriting each other
  - Tracking exactly what changed between draft versions
  - Automatic replication audit trail
- The good news: Claude Code can handle all Git commands for you — you just describe what you want

Git Integration

Key Git Concepts (No Commands Required)

- **Repository (repo)**: a folder whose history Git is tracking
- **Commit**: a snapshot of your code at a point in time, with a message
- **Push / Pull**: sync your local repo with GitHub (the remote copy)
- **Branch**: a parallel version of the code for a new feature or robustness check
- **Merge**: fold a branch back into the main code
- Typical workflow for an economist:
  1. Work on code; when something works, commit it
  2. Push to GitHub at end of day (backup + coauthor access)
  3. RA works on a branch; you review and merge

Git and Claude Code: What the Integration Looks Like

- ▷ *Placeholder: screenshot or demo*
- In VS Code: Git panel shows all changes; Claude Code can stage, commit, push on request
- You tell Claude: “commit my changes with a descriptive message” — it writes the message too
- Branching for research: create a branch for each robustness check, merge if it works
- Connecting with Overleaf:
  - Overleaf Pro can sync with a GitHub repo
  - Alternatively: LaTeX Workshop extension in VS Code + local compilation

Python for Economists

Python in VS Code: Environments

- ▷ *Demo slide — Tanner*
- Anaconda manages Python installations and packages
- Each project gets its own **environment**: a fixed set of packages and versions
  - Prevents conflicts between projects
  - Ensures others can reproduce your results exactly
- Ask Claude Code to create and activate an environment for you

Running Python Code with Jupyter

- ▷ *Demo slide — Tanner*
- Three modes in VS Code:
  - **Jupyter notebook** (`.ipynb`): interactive cells, inline output — good for exploration
  - **Interactive window**: run `.py` scripts cell-by-cell — best of both worlds
  - **Script** (`.py`): run the full pipeline — best for production code
- Claude Code can write, run, and debug Python in any of these modes

Setting Up Your Environment

What You Need to Install

- Core tools (all free except AI subscription):
  1. **VS Code** — the editor (free)
  2. **Claude Code** — VS Code extension + CLI (requires subscription)
  3. **Git** — version control (free); create a GitHub account
  4. **Python** — Anaconda distribution recommended (free)
- VS Code extensions to install:
  - Claude Code, GitLens, Jupyter, Python, Stata Enhanced (if applicable)
- We have a step-by-step setup guide — see handout / shared folder

Session 1 — Key Takeaways

1. AI has perhaps most value added in coding
2. Agentic $\neq$ chatbot: reads files, writes and runs code, fixes errors, reports back
3. Two serious agentic options today: Claude Code and Codex
   - Pick one, learn it, reassess in 6 months
4. Tokens accumulate fast in long session — start new session for each new task
5. Three interfaces (Terminal, Desktop, VS Code) and Git are your essential stack

Questions?

- Questions on tools, concepts, or setup?
- Thoughts on how this fits your current workflow?